On the Existence of EFX (and Pareto-Optimal) Allocations for Binary Chores

Biaoshuai Tao[1*] **Xiaowei Wu**[2], **Ziqi Yu**[1], **Shengwei Zhou**[2]

[1]Shanghai Jiaotong University, {bstao, yzq.lll}@sjtu.edu.cn
[2]University of Macau, {xiaoweiwu, yc17423}@um.edu.mo

Abstract

We study the problem of allocating a group of indivisible chores among agents while each chore has a binary marginal. We focus on the fairness criteria of envy-freeness up to any item (EFX) and investigate the existence of EFX allocations. We show that when agents have additive binary cost functions, there exist EFX and Pareto-optimal (PO) allocations that can be computed in polynomial time. To the best of our knowledge, this is the first setting of a general number of agents that admits EFX and PO allocations, before which EFX and PO allocations have only been shown to exist for three bivalued agents. We further consider more general cost functions: cancelable and general monotone (both with binary marginal). We show that EFX allocations exist and can be computed for binary cancelable chores, but EFX is incompatible with PO. For general binary marginal functions, we propose an algorithm that computes (partial) envy-free (EF) allocations with at most $n-1$ unallocated items.

1 Introduction

The *fair division* problem mainly focuses on allocating heterogeneous resources fairly to a group of agents. It is a classic resource allocation problem that has been widely studied by mathematicians, economists, and computer scientists. In this research direction, a popular question is to focus on allocating *indivisible* items. An ideal fairness notion is *envy-freeness* (EF) (Foley 1967), which requires that every agent values her own bundle not lower than any other bundle held by others. However, EF allocations are not guaranteed to exist when allocating indivisible items, e.g. allocating one item to two agents. Caragiannis et al. (2019) proposed a relaxed notion called *envy-freeness up to any item* (EFX), which requires that for any pair of agents i and j, i does not envy j after removing an arbitrary item from j's bundle. The existence of EFX allocations still remains open for general instances and is only known for some special cases, e.g., two agents with general valuations (Plaut and Roughgarden 2020), three agents with some special valuations (Akrami et al. 2023; Chaudhury, Garg, and Mehlhorn 2020) and *bivalued* instances (Amanatidis et al. 2021).

In the traditional setting of fair division problems, items are assumed to have non-negative effects on agents. Roughly speaking, giving an item to any agent will make her happier. This kind of question is called fair division of *goods*. On the contrary, the complement setting is called fair division of *chores*. In this case, each item is of negative value to agents, and we use *cost* functions to describe the preference of each agent. Some definitions of fairness criteria (such as EFX) can be naturally extended to chores division (Aziz et al. 2022). However, most of the known results do not directly extend to the chores setting. The minor differences between goods and chores settings nullify some existing techniques, such as Cycle-Elimination, when considering the allocation of chores (Zhou and Wu 2022). Another example can be given by the comparison of goods and chores on fair and efficient allocations. As one of the influential results in the world of goods, Caragiannis et al. (2019) showed that maximizing *Nash social welfare* guarantees the fairness of *envy-freeness up to one item* (EF1) (a fairness notion weaker than EFX) and the efficiency of Pareto-optimality (PO). However, the existence of EF1 and PO allocations remains a major open problem in the context of chores, except for a limited class of bi-valued additive cost functions (Garg, Murhekar, and Qin 2022a; Ebadian, Peters, and Shah 2022).

Major of our results focus on the settings when the agents' costs for chores have *binary marginals*. To be exact, for a binary cost function c, $c(S \cup \{g\}) - c(S) \in \{0, 1\}$, where S is any set of chores and g is a single chore. Binary marginals have received much attention in the literature for both goods (Babaioff, Ezra, and Feige 2021; Barman and Verma 2021; Kurokawa, Procaccia, and Shah 2018) and chores (Barman, Narayan, and Verma 2023), due to various real-world applications such as bilateral matching (Bogomolnaia and Moulin 2004) and housing allocations (Benabbou et al. 2020). The binary marginal gives us extra properties to develop new techniques.

Our results

In this paper, we study the fair (and efficient) allocations of indivisible chores when each chore has a binary marginal cost. We consider the fairness notion of envy-freeness up to any item (EFX) and investigate the existence of EFX allocations for different cost functions. For additive cost functions with binary marginal, we show that EFX and Pareto-optimal (PO) allocations exist and can be computed in polynomial

[*]The authors are ordered alphabetically.

time. Prior to our result, it has only been shown that EFX and PO allocations exist for restricted instances with three bivalued agents (Garg, Murhekar, and Qin 2022b). This is the first setting, even for additive cost functions, that admits EFX and PO allocations for a general number of agents.

Result 1 (Theorem 3.1) **.** There exists a polynomial time algorithm that computes EFX and PO allocations for additive cost functions with binary marginals.

We further consider the *cancelable* cost functions (with binary marginals) that are first introduced by Berger et al. (2022). Cancelable cost functions generalize several cost functions including *additive*, *budget additive*, and more. We show that EFX is no longer compatible with PO for the more general cancelable cost functions. We propose a polynomial-time algorithm that computes EFX allocations for binary cancelable cost functions. To the best of our knowledge, this is the first non-trivial result regarding the existence and computation of EFX allocations for non-identical cost functions that are beyond additive.

Result 2 (Theorem 4.1) **.** There exists a polynomial time algorithm that computes EFX allocations for cancelable cost functions with binary marginals.

We further consider submodular and general cost functions with binary marginals. We show that under the binary marginal property, the set of submodular cost functions is a superset of the set of cancelable cost functions. We propose a polynomial-time algorithm that computes 2-approximately EFX allocations for binary submodular cost functions, and (partial) EF allocations with at most $n-1$ unallocated items for general cost functions with binary marginals.

Result 3 (Theorem 5.1) **.** There exists a polynomial time algorithm that computes a partial allocation that is envy-free and leaves at most $n-1$ items unallocated, for general cost functions with binary marginals.

Result 4 (Theorem D.1, Corollary D.2) **.** There exists a polynomial time algorithm that computes 2-approximately EFX allocations for submodular cost functions with binary marginals.

We summarize our results in Figure 1 which also presents the relationship between different cost functions with binary marginals.

Other Related Work

The setting of binary marginal cases is widely considered in recent years. In the literature of goods, Babaioff, Ezra, and Feige (2021) and Halpern et al. (2020) considered the class of binary submodular (or matroid-rank) valuations and showed that EFX and PO allocations always exist. They designed a mechanism that maximizes *Nash social welfare* which generates EFX and PO properties under the matroid-rank valuations. Viswanathan and Zick (2023) improved this by proposing the "Yankee-Swap algorithm" that runs significantly faster. However, maximizing Nash social welfare might fail to guarantee EFX allocations when it comes to more general valuations, which makes it impossible to extend Babaioff et al.'s and Halpern et al.'s mechanisms directly. Very recently, Bu, Song, and Yu (2023) developed a polynomial-time algorithm, based on cycle-elimination techniques, that computes EFX allocations for general valuations with binary marginals. However, these results can not be easily extended to that of chores. In the setting of chores division, the cycle-elimination process will sometimes break the EFX property, which will not happen in the world of goods division. When it comes to binary chores division, Barman, Narayan, and Verma (2023) considered binary supermodular costs and showed that: 1) EF1 and PO allocations exist and can be computed in polynomial time; 2) EFX and PO are incompatible, under the binary supermodular valuations. However, the existence of EFX allocations for non-identical cost functions and the compatibility of EFX and PO for other binary functions, e.g., *additive* and *submodular*, are still open.

Though the existence of EFX allocations remains open, much partial progress has been made over the past years. A line of the literature focuses on exploring under which restriction EFX allocations exist. For the allocation of goods, Plaut and Roughgarden (2020) showed that EFX allocations exist when all agents have identical valuations. On the restriction of the number of agents, EFX allocations have been shown to exist for two agents with general valuations (Plaut and Roughgarden 2020) and three agents with additive valuations (Chaudhury, Garg, and Mehlhorn 2020). The latter result was recently extended to more general valuations by Akrami et al. (2023). In the context of chores, it has been shown that EFX allocations exist for some special cases, e.g., agents have monotone identical functions (Bérczi et al. 2020), identical ordering (IDO) instances (Li, Li, and Wu 2022), three agents with bivalued additive functions (Zhou and Wu 2022), two types of chores (Aziz et al. 2023). Another branch of the literature is to explore the relaxations of EFX, e.g., approximately EFX allocations (Plaut and Roughgarden 2020; Chan et al. 2019; Amanatidis, Markakis, and Ntokos 2020; Zhou and Wu 2022) and partial EFX allocations with reasonable guarantees (Chaudhury et al. 2021b; Berger et al. 2022; Caragiannis, Gravin, and Huang 2019; Chaudhury et al. 2021a; Jahan et al. 2022). For a more comprehensive overview of the fair division problem, we refer to the recent surveys by Amanatidis et al. (2023) and Aziz et al. (2022).

2 Preliminaries

We consider how to fairly allocate a set of m indivisible chores/items M to a group of n agents N. We call a subset of items, e.g., $S \subseteq M$, a *bundle*. For ease of notation we use $X+e$ and $X-e$ to denote $X \cup \{e\}$ and $X \setminus \{e\}$, respectively, for any $X \subseteq M$ and $e \in M$. A complete allocation $\mathbf{X} = (X_1, \ldots, X_n)$ is an n-partition of the items M such that $X_i \cap X_j = \emptyset$ for all $i \neq j$ and $\cup_{i \in N} X_i = M$, where agent i receives bundle X_i. When $\cup_{i \in N} X_i \subsetneq M$, we call it a partial allocation. Given an instance $I = (N, M, \mathbf{c})$ where $\mathbf{c} = (c_1, \ldots, c_n)$ is the set of *cost functions* (to be defined immediately), our goal is to find an allocation $\mathbf{X}$ that is *fair* to all agents.

Cost Functions. Each agent $i \in N$ has a cost function $c_i : 2^M \to \mathbb{R}^+ \cup \{0\}$ that assigns a cost to every bundle of items. More specifically, for any subset of items $S \subseteq M$,

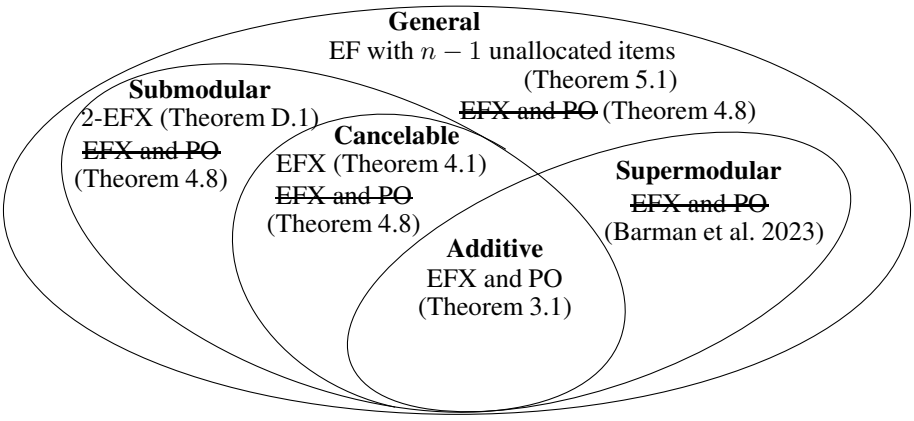

Figure 1: Illustration of our results and the relationship between different set functions with binary marginals

the cost of S to agent i is denoted as $c_i(S)$. For convenience we use $c_i(e)$ to denote $c_i(\{e\})$, the cost of agent $i \in N$ on item $e \in M$. We use $\mathbf{c} = (c_1, \ldots, c_n)$ to denote the cost functions of agents. For any subset of items $S \subseteq M$, any item $e \in M$, we use $c_i(e \mid S)$ to denote the marginal cost of item e to subset S, under agent i's cost function, i.e., $c_i(S + e) - c_i(S)$. Similarly, for $S, T \subseteq M$, we denote $c_i(T \mid S) = c_i(T \cup S) - c_i(S)$.

- *Binary Marginal.* We focus on the instances in which each item gives *binary* marginal cost to any subset of items, that is, for any $i \in N, e \in M, S \subseteq M$, we have $c_i(e \mid S) \in \{0, 1\}$. Note that, any binary marginal cost function is also *monotone*, i.e., $c_i(S+e) \geq c_i(S)$ for any $i \in N, e \in M, S \subseteq M$.
- *Additive Function.* A cost function $c_i(\cdot)$ is said to be additive if $v_i(S) = \sum_{e \in S} c_i(e)$ for any $S \subseteq M$.
- *Cancelable Function.* As a generalization of additive cost functions, cancelable functions are first introduced by Berger et al. (2022) in the fair allocation of goods. A cost function $c_i(\cdot)$ is said to be cancelable if for any two bundles $S, T \subseteq M$ and item $e \in M \setminus (S \cup T)$, we have

$$c_i(S + e) > c_i(T + e) \Rightarrow c_i(S) > c_i(T). \quad (1)$$

  As Berger et al. (2022) pointed out, cancelable functions describe many natural meaningful valuation functions in economics, including additive, unit-demand, and budget-additive cost functions.
- *Submodular Function.* A cost function $c_i(\cdot)$ is submodular if and only if for any $S \subseteq T, e \in M \setminus T$, we have $c_i(e \mid S) \geq c_i(e \mid T)$.

In Appendix A, we will show that, under set functions with binary marginals, the set of additive functions is a proper subset of the set of cancelable functions, and the set of cancelable functions is a proper subset of the set of submodular functions. Notice that the latter only holds for set functions with binary marginals.

Fairness. We first introduce the *envy-freeness* (EF) for the allocation of chores. An allocation $\mathbf{X}$ is EF if for any agents $i, j \in N$, $c_i(X_i) \leq c_i(X_j)$. Since EF allocations are not guaranteed to exist for indivisible chores, we focus on a relaxation of EF: *envy-freeness up to any item* (EFX). An allocation $\mathbf{X}$ is EFX if for any agents $i, j \in N$, either $X_i = \emptyset$, or $c_i(X_i - e) \leq c_i(X_j)$ for any item $e \in X_i$. For $\alpha \geq 1$, an allocation is α-approximately envy-free, denoted by α-EF, if $c_i(X_i) \leq \alpha \cdot c_i(X_j)$; an allocation is α-approximately EFX, denoted by α-EFX, if either $X_i = \emptyset$ or $c_i(X_i - e) \leq \alpha \cdot c_i(X_j)$ for any $i, j \in N$ and any $e \in X_i$.

Efficiency. An allocation $\mathbf{X}'$ *Pareto-dominates* another allocation $\mathbf{X}$ if $c_i(X'_i) \leq c_i(X_i)$ for all $i \in N$ and the inequality is strict for at least one agent. An allocation $\mathbf{X}$ is said to be *Pareto-optimal* (PO) if $\mathbf{X}$ is not Pareto-dominated by any other allocation. For any allocation $\mathbf{X} = (X_1, \cdots, X_n)$, the social cost $\mathrm{sc}(\mathbf{X})$ is defined as the sum of the cost of each agent, i.e., $\mathrm{sc}(\mathbf{X}) = \sum_{i \in N} c_i(X_i)$. An allocation that minimizes the social cost is naturally Pareto-optimal since any Pareto-improvement strictly decreases the social cost.

Relationships Between Set Functions

In Appendix A, we prove the following two theorems, which state that additive functions form a special case of cancelable functions, and cancelable functions form a special case of submodular functions, when we are considering functions with binary marginals. Notice that Theorem 2.2 only holds for functions with binary marginals.

Theorem 2.1. *The set of binary additive set functions is a proper subset of the set of cancelable set functions with binary marginals.*

Theorem 2.2. *The set of cancelable set functions with binary marginals is a proper subset of the set of submodular set functions with binary marginals.*

We also list some properties for cancelable and submodular functions.

Proposition 2.3. *Let $\phi : M \to \mathbb{Z}_{\geq 0}$ be a cancelable function.*

1. *For any $S, T \subseteq M$ and any $e \in M \setminus (S \cup T)$, if $\phi(S) = \phi(T)$, then $\phi(S + e) = \phi(T + e)$.*
2. *For any $S, T \subseteq M$ and any $U \subseteq M$ with $U \cap (S \cup T) = \emptyset$, if $\phi(S) = \phi(T)$, then $\phi(S \cup U) = \phi(T \cup U)$.*

Proposition 2.4. *Let $\phi : M \to \mathbb{Z}_{\geq 0}$ be a submodular function with binary marginals.*

1. *For any $S, T \subseteq M$, $\phi(S)+\phi(T) \geq \phi(S \cup T)+\phi(S \cap T)$. In particular, $\phi(S) + \phi(T) \geq \phi(S \cup T)$.*
2. *If $\phi(S) = 1$ holds for some S with $|S| \geq 2$, there exists $e \in S$ such that $\phi(S - e) = 1$.*

3 Existence of EFX and PO Allocations

In this section, we explore the existence of EFX and PO allocations for additive cost functions. We call an instance *binary* if each agent has an additive valuation with binary marginals. We propose an algorithm that computes EFX and PO allocations for binary instances. This is the first class with a general number of agents, for which EFX and PO allocations exist and can be computed in polynomial time. In the world of allocation of goods, it has been shown that maximizing Nash social welfare (product of agents' utilities) leads to EFX and PO allocations for bivalued instances (which is a superset of binary instances) (Amanatidis et al. 2021; Garg and Murhekar 2021). However, in the opposite of goods, minimizing Nash social welfare results in zero Nash social welfare which has no guarantee of fairness and efficiency. Before our result, it has only been shown that EFX and PO allocations exist for three bivalued agents (Garg, Murhekar, and Qin 2022b).

Given a binary instance, we divide the set of chores into M^0 and M^+, while M^+ includes those items that cost 1 to all agents, i.e., $c_i(e) = 1$ for all $i \in N$.

$$M^0 = \{e \in M : \exists i \in N, c_i(e) = 0\},$$
$$M^+ = \{e \in M : \forall i \in N, c_i(e) = 1\}.$$

In this section, we prove the following main result.

Theorem 3.1. *There exists an algorithm that computes EFX and PO allocations for any given binary instance in polynomial time.*

Our algorithm starts from a partial allocation $\mathbf{X}^0$ in which every agent only receives items that have zero cost to her. In the second phase, we allocate each item $e \in M^+$ to one agent straightly if it maintains EFX property after allocating e, or we reallocate some items until an item $e \in M^+$ can be allocated without breaking EFX. The steps of the full algorithm are summarized in Algorithm 1.

Lemma 3.2. *Algorithm 1 returns an allocation $\mathbf{X}$ with minimum social cost.*

Proof. We prove the lemma by showing that $\mathrm{sc}(\mathbf{X}) = |M^+|$, while no complete allocation has a strictly less social cost. In Phase 1 we compute a partial allocation X^0 with zero social cost and remaining all items in M^+ unallocated. Then in Phase 2 we allocate items in M^+ and reallocate some items, while each allocation of item $e \in M^+$ increases the total social cost by 1. Note that item $e \in M$ would be reallocated to agent i only under the case that $c_i(e) = 0$. Hence any reallocation would not change the total social cost of the allocation. In conclusion, for the returned allocation $\mathbf{X}$ we have $\mathrm{sc}(\mathbf{X}) = |M^+|$. □

Lemma 3.3. *The allocation $\mathbf{X}$ returned by Algorithm 1 is EFX to all agents.*

Algorithm 1: Finding an EFX and PO allocation for binary additive cost functions

Input: A binary instance $< M, N, \mathbf{c} >$
1 initialize $X_i \leftarrow \emptyset$ for all $i \in N$, $P \leftarrow M$;
// Phase 1: compute a partial allocation $\mathbf{X}^0$ with $\mathrm{sc}(\mathbf{X}^0) = 0$.
2 **for** *each item* $e \in M^0$ **do**
3 pick an agent $i \in N$ such that $c_i(e) = 0$. break tie arbitrary;
4 update $X_i \leftarrow X_i + e$;
// Phase 2: allocate items in M^+.
5 initialize $P \leftarrow M^+$;
6 **while** $P \neq \emptyset$ **do**
7 let $i^* \leftarrow \mathrm{argmin}_{i \in N}\{c_i(X_i)\}$, break tie arbitrary;
8 update $X_{i^*} \leftarrow X_{i^*} + e$, $P \leftarrow P - e$;
9 **if** *there exists an agent $j \neq i^*$ such that i^* is not EFX towards j* **then**
10 update $X_{i^*} \leftarrow X_{i^*} - e, X_j \leftarrow X_j + e$;
11 **while** *there exists an item $e' \in X_j$ such that* $c_{i^*}(e') = 0$ **do**
12 update $X_{i^*} \leftarrow X_{i^*} + e', X_j \leftarrow X_j - e'$;
Output: $\mathbf{X} = \{X_1, \cdots, X_n\}$.

Proof Sketch We show that the allocation $\mathbf{X}$ is EFX by induction and treat the beginning of Phase 2 as the basic case. The statement holds for the basic case since $c_i(X_i) = 0$ for all $i \in N$. Then we assume it holds at the beginning of some round t of Phase 2 and we show that it still holds at the end of round t. According to the algorithm, we only need to consider the case under the if-condition in line 9. As for agent i, the envy between i and other agents almost stays the same since agent i only receives some items that cost 0 to her. Then we consider the envy between j and other agents. An important claim we used is that $c_i(X_i) = c_j(X_j)$ holds when the if-condition from Line 9 to Line 12 is executed, and $X_j \setminus S \subseteq M^+$ holds after the execution. The EFX property stays trivially for any agent rather than j. The allocation is EFX for agent j since after removing any item, she holds the minimum bundle over all agents.

Furthermore, it can be easily verified that the algorithm runs in polynomial time. We further complete our result by exploring the non-existence of EFX and PO allocations in the additive setting. We show that even extending our result to ternary instances[1] is impossible (see Appendix B).

4 EFX Allocations Exist for Binary Cancelable Chores

In this section, we show that, for cancelable cost functions with binary marginals, we can always find EFX allocations in polynomial time, but there exist instances where all EFX allocations are not Pareto-optimal.

[1] An instance is called ternary if all cost functions are additive and $c_i(e) \in \{0, 1, 2\}$ for all $i \in N, e \in M$.

Theorem 4.1. *For cancelable cost functions with binary marginals, EFX allocations always exist and can be computed in polynomial time.*

Algorithm 2: Finding an EFX allocation for cancelable cost functions with binary marginals

Input: A binary instance $(M, N, \mathbf{c})$

`// Phase 1: allocate items with marginal cost 1 from all agents' perspective`

initialize $(A_1, \dots, A_n)$ with $A_i = \emptyset$ for all i;

while *there exist n items such that each item e satisfies $c_i(e \mid A_i) = 1$ for all i* **do**

  add those n items to $(A_1, \dots, A_n)$ such that each A_i is added one item;

let $w = |A_1| = \cdots = |A_n|$;

`// Phase 2: allocate the remaining items`

initialize $(B_1, \dots, B_n)$ where $B_i = \emptyset$ for each i;

consider the cost function $\mathbf{d} = (d_1, \dots, d_n)$ with $d_i(S) = c_i(S \mid A_i)$;

let $M_1 = \{e \in M : d_i(e) = 1 \text{ for all } i\}$;

`// We have` $|M_1| < n$`, for otherwise Phase 1 should not have been terminated.`

for each $i = 1, \dots, |M_1|$, let B_i be the set containing one (arbitrary) item of M_1;

while *there is an unallocated item e* **do**

  `// We maintain that` $(B_1, \dots, B_n)$ `is EFX with respect to` $\mathbf{d}$ `and` $d_i(B_i) \le 1$ `for any` i`.`

  if $d_i(e \mid B_i) = 0$ *for some agent i and* $(B_1, \dots, B_n)$ *is EFX after updating* $B_i \leftarrow B_i + e$ **then**

    update $B_i \leftarrow B_i + e$;

  else

    `// If we reach here, there must exist` i `and` j`, with the possibility` $i = j$`, such that` $d_i(B_j) = 0$ `(Proposition 4.6).`

    if *there exists $i \in N$ with $d_i(B_i) = 0$* **then**

      if there exists $j \neq i$ with $d_i(B_j) = 0$, update $B_i \leftarrow B_i \cup B_j$ and $B_j \leftarrow \{e\}$; otherwise, update $B_i \leftarrow B_i + e$;

    else

      find j such that $d_i(B_j) = 0$, and swap B_i and B_j;

Output: $\mathbf{X} = (X_1, \dots, X_n)$ where $X_i = A_i \cup B_i$ for each i.

Our algorithm is presented in Algorithm 2. The algorithm consists of two phases. In the first phase, we iteratively allocate n items, where each agent is believed to have a marginal cost of 1, such that each agent is allocated exactly one item. Let w be the number of items allocated to each agent, and we have allocated wn items. After Phase 1, the number of items with marginal costs 1 to all agents is then less than n. Let M_1 be the set of these items. Let $(A_1, \dots, A_n)$ be the allocation of Phase 1. After the first phase, each agent believes that her own bundle costs w and each other agent's bundle also costs w, as the lemma below states.

Lemma 4.2. *After Phase 1, we have $c_i(A_j) = w$ for each pair of i and j, with the possibility $i = j$.*

Proof. This can be proved by straightforward induction. See Appendix C for the full proof. □

In Phase 2, we compute an allocation $(B_1, \dots, B_n)$ of the remaining items. We update the cost function c_i such that, for each unallocated set S of items, the cost of S is given by $c_i(S \mid A_i)$. Let $\mathbf{d}$ be the set of the updated cost functions. We will show that the allocation $(B_1, \dots, B_n)$ output in Phase 2 satisfies the following two key properties:

- $d_i(B_i) \le 1$ for each agent i.
- $(B_1, \dots, B_n)$ is EFX with respect to $\mathbf{d}$.

Lastly, the final allocation $\mathbf{X} = (X_1, \dots, X_n)$ is given by $X_i = A_i \cup B_i$ for each $i \in N$. We will show that $\mathbf{X}$ is EFX with respect to $\mathbf{c}$. The cancelable property plays an important role in guaranteeing EFX property when combining the allocations in the two phases. Specifically, the cancelable property and Lemma 4.2 ensure a good interpolation between the two cost function sets $\mathbf{d}$ and $\mathbf{c}$.

Before proving the key properties of Phase 2 allocation, we first state the following proposition whose proof is straightforward.

Proposition 4.3. *Each d_i is a cancelable and submodular set function with binary marginals.*

Now we are ready to show our main lemma for Phase 2. It states that the algorithm will always be terminated and the allocation output satisfies our two key properties.

Lemma 4.4. *The while-loop in Phase 2 is executed for at most $2m$ iterations, and the output allocation $(B_1, \dots, B_n)$ satisfies that*

1. *$d_i(B_j) \le 1$ for each agent i, and*
2. *$(B_1, \dots, B_n)$ is EFX with respect to $\mathbf{d}$.*

We first show that Property 1 and 2 above are always satisfied.

Proposition 4.5. *After any number of while-loop iterations of Phase 2, the two properties 1 and 2 in Lemma 4.4 are satisfied.*

Proof. Before entering the while-loop, exactly $|M_1|$ bundles contain one item with cost 1 from all agent's perspectives, and the remaining $n - |M_1|$ bundles are empty. The properties clearly hold. Next, suppose the properties hold before a while-loop execution, we will show that they continue to hold after. We will check all the updates, at Line 11, 14, and 16, do not invalidate the properties.

The update at Line 11 adds an item with marginal cost 0 to the bundle B_i, so Property 1 continues to hold. It will not invalidate Property 2 by the if-condition.

For the update at Line 14, we have $d_i(B_i \cup B_j) \le d_i(B_i) + d_i(B_j) = 0$ by submodularity of d_i (Proposition 4.3), $d_i(B_i + e) \le 1$, and $d_j(e) \le 1$. In all cases, Property 1 continues to hold. To check Property 2, if there does

not exist $j \neq i$ with $d_i(B_j) = 0$, then agent i receives $B_j + e$ and the allocation for the remaining agents is unchanged. In this case, $d_i(B_j) \geq 1$, and agent i does not envy any other agent by receiving $B_i + e$, which has cost at most 1. The EFX condition between any other agent and i still holds as i receives a superset of B_i. If $d_i(B_j) = 0$ for some j, agent i receives $B_i \cup B_j$ and agent j's bundle is updated to the singleton $\{e\}$. We have $d_i(B_i \cup B_j) \leq d_i(B_i) + d_i(B_j) = 0$, so i does not envy any other agent. On the other hand, j receives only a single item e, she will no longer envy any other agent if this item were removed. Consider any other agent k that is neither i nor j. The EFX condition between k and i continues to hold, as i now receives a superset of B_i. As for the EFX condition between k and j, we discuss two cases. If $d_k(e) = 1$, then agent k does not envy agent j as we have $d_k(B_k) \leq 1$ by Property 1. If $d_k(e) = 0$, we have $d_k(e \mid B_k) \leq d_k(e) = 0$ by submodularity. Since the if-condition at Line 10 fails (otherwise, the "else" part will not be executed), adding e to B_k will break the EFX condition between k and some other agent ℓ. By Property 1, it must be that $d_k(B_k) = 1$ and $d_k(B_\ell) = 0$. Moreover, $d_k(B_k - f) = 0$ for any $f \in B_k$ (in fact, it must be that $B_k = \{f\}$ by Property 2 in Proposition 2.4). In this case, the EFX condition between k and any other agent still holds.

For the update at Line 16, if we have ever reached here, it must be that $d_i(B_i) = 1$ and $d_i(B_j) = 0$. Moreover, we must have $|B_i| = 1$. Otherwise, by Property 2 of Proposition 2.4, the EFX property between i and j will be violated. After the update at Line 16, we have $d_i(B_i) = 0$ and $d_j(B_j) \leq 1$, and the latter holds because B_j, which is previously B_i, contains only one item. This proves Property 1. To check Property 2, i does not envy any other agents as $d_i(B_i)$ is now 0. The EFX condition between j and any other agents holds as j's bundle contains only one item now. The EFX condition between any other agent k and i or j still holds as we have only swapped the bundles of i and j. □

To show the algorithm terminates, we prove the following observation which is also stated between Line 12 and Line 13 of the algorithm.

Proposition 4.6. *If the "else" part from Line 12 to Line 16 is executed, there must exist i and j, with the possibility $i = j$, such that $d_i(B_j) = 0$.*

Proof. Suppose $d_i(B_j) \geq 1$ for every pair of i and j. We will show that the if-condition at Line 10 is always satisfied. Proposition 4.5 indicates that Property 1 in Lemma 4.4 holds before the if-condition at Line 10. Therefore, the current allocation $(B_1, \ldots, B_n)$ is envy-free. Adding an item with a zero marginal cost to an agent will not destroy the envy-free property. On the other hand, for each $e \notin M_1$, there exists an agent i with $d_i(e) = 0$, and, by submodularity, $d_i(e \mid B_i) = 0$. Since all the items in M_1 have been allocated before the while-loop and the algorithm never returns an item back to the pool of unallocated items, the if-condition at Line 10 will be satisfied. □

Now we are ready to prove Lemma 4.4.

Proof of Lemma 4.4. Proposition 4.5 ensures that the two properties always hold. It remains to show that the while-loop will be executed for at most $2m$ iterations. It suffices to show that at least one item is allocated in every two iterations. The only case where no item is allocated is when Line 16 is executed. In this case, Proposition 4.6 ensures the existence of j. After the swapping, we have $d_i(B_i) = 0$. Therefore, in the next iteration, either Line 11 or Line 14 will be executed, in which case an item is allocated. □

Remark 4.7. *For the proof of Lemma 4.4, we have only exploited the submodularity of d_i, not the cancelability. In particular, if no item is allocated in Phase 1 and the algorithm starts with Phase 2, we have $\mathbf{d} = \mathbf{c}$. The algorithm computes an EFX allocation even if each c_i is only known to be submodular (while not necessarily cancelable).*

Finally, we combine Lemma 4.2 and Lemma 4.4 to conclude Theorem 4.1.

Proof of Theorem 4.1. We first show that the allocation output by Algorithm 2 is EFX. We check the EFX condition for an arbitrary pair of agents i and j. By Lemma 4.4, $(B_1, \ldots, B_n)$ is EFX with respect to $\mathbf{d}$, and we consider two cases: 1) $d_i(B_i) \leq d_i(B_j)$ and 2) $d_i(B_i) > d_i(B_j)$.

For the first case, we have

$$\begin{aligned} c_i(X_i) = c_i(A_i) + d_i(B_i) &\leq c_i(A_i) + d_i(B_j) && \text{(case assumption)} \\ &= c_i(A_i) + (c_i(A_i \cup B_j) - c_i(A_i)) = c_i(A_i \cup B_j) && \text{(definition of } d_i\text{)} \\ &= c_i(A_j \cup B_j) = c_i(X_j), && \text{(Property 2 of Proposition 2.3 and Lemma 4.2)} \end{aligned}$$

which indicates that i does not envy j.

For the second case, it must be that $|B_i| = 1$. To see this, Property 1 of Lemma 4.4 indicates that $d_i(B_i) = 1$ and $d_i(B_j) = 0$. Property 2 of Proposition 2.4 indicates that $|B_i| \geq 2$ will destroy the EFX property (Property 2 of Lemma 4.4) between i and j. Since we have seen $|B_i| = 1$, $|X_i| = w + 1$. Therefore, agent i's cost will be at most w after removing any item from X_i. On the other hand, by Lemma 4.2 and the monotonicity, $c_i(X_j) \geq c_i(A_j) = w$. Thus, the EFX condition between i and j is satisfied.

Lastly, checking that the algorithm runs in polynomial time is straightforward. □

We have seen that EFX allocations always exist for binary cancelable chores. However, unlike the case with additive cost functions, EFX is no longer compatible with PO (see Appendix C). This is in sharp contrast to the setting with goods. For allocating goods, we know that EFX is compatible with PO even for the more general submodular binary cost functions: Babaioff, Ezra, and Feige (2021) show that the allocation maximizing the Nash social welfare (product of agents' utilities) is EFX.

Theorem 4.8. *For any number of agents $n \geq 2$, there exist instances with binary cancelable cost functions where all EFX allocations are not Pareto-optimal.*

5 General Cost Functions with Binary Marginals

For general cost functions with binary marginals, we show that there exists a partial allocation that leaves at most $n-1$ items unallocated and is envy-free. We will also show that the algorithm can be used to find a complete 2-EFX allocation for submodular binary cost functions (we defer this part to Appendix D due to the page limit). We remark that we do not know if complete EFX allocations always exist, even for the more special case with submodular binary cost functions.

Theorem 5.1. *For cost functions with binary marginals, there exists a partial allocation that is envy-free and leaves at most $n-1$ items unallocated. Moreover, such an allocation can be computed in polynomial time.*

The algorithm makes use of the *envy graph*, a technique that has been widely used in the fair division literature.

Definition 5.2. *Given a partial allocation $(X_1, \ldots, X_n)$ and the cost function profile* $\mathbf{c}$*, the* envy graph $G = (V = [n], E)$ *is a directed graph where each vertex in V represents an agent and $(i,j) \in E$ if and only if $v_i(A_i) = v_i(A_j)$.*

As a remark, unlike it is in most of the previous literature where an edge represents "envy", we always maintain an envy-free allocation and an edge in our envy graph represents equality, or, "about to envy".

Algorithm description and proof of Theorem 5.1. Our algorithm is presented in Algorithm 3. It initializes the allocation with n empty bundles, and the initial envy graph is a complete graph. At each iteration, it attempts to allocate one or more items by using one of the three update rules:

1. if there is an item with marginal cost 0 to some agent, allocate it;
2. if an edge (i,j) is on a cycle C and i thinks adding some item e to j's bundle does not increase its cost, rotate the bundles on the cycle so that i receive j's bundle, and add e to i's bundle (which is previously j's) which does not increase the cost for i;
3. if the first two update rules do not apply, find a tail strongly connected component S (a strongly connected component with no outgoing edges) and allocate each agent in S an arbitrary item; if the number of items is insufficient for this, the algorithm is terminated with the partial allocation outputted.

We will show that the EF property is satisfied throughout the algorithm. The initial allocation is clearly envy-free. It suffices to show that, given a partial allocation, any of the three update rules does not destroy envy-freeness.

It is straightforward to check the first and the second update rules do not invalidate envy-freeness.

For the third update rule, first notice that the envy-freeness between an agent i in S and an agent k in $V \setminus S$ is preserved. Agent k does not envy agent i as agent i's cost can only be increased from k's perspective. Since S is a tail strongly connected component, $(i,k) \notin E$ before the update, so $v_i(X_i) \leq v_i(X_k) - 1$. Adding an item to agent i, which increases $v_i(X_i)$ by at most 1 (in fact, it is exactly 1, for otherwise the first update rule should be applied), does not make i envy k.

Now, consider any two agents $i, j \in S$. If $(i,j) \notin E$ before the update, i will not envy j after the update even in the worst case where $v_i(X_i)$ is increased by 1 and $v_i(X_j)$ is unchanged. If $(i,j) \in E$ before the update, (i,j) is on a cycle by the property of strongly connected components. Since the first two update rules do not apply, it must be that $c_i(e \mid X_i) = c_i(e \mid X_j) = 1$ for any unallocated item e. Adding an item to i and an item to j increases both $c_i(X_i)$ and $c_i(X_j)$ by 1. Envy-freeness is again preserved.

Finally, checking that the algorithm runs in polynomial time is straightforward. We can only reach a partial allocation when the if-condition at Line 12 fails, in which case the number of unallocated items is less than $|S|$, which is at most $n-1$.

Algorithm 3: Finding a partial EF allocation for cost functions with binary marginals

Input: A binary instance $(M, N, \mathbf{c})$

1 initialize $(X_1, \ldots, X_n)$ with n empty bundles, and initialize the envy graph $G = (V, E)$;
2 **while** *there exist unallocated items* **do**
3 update the envy graph $G = (V, E)$;
4 **if** *there exist an unallocated item e and i with $c_i(e \mid X_i) = 0$* **then**
5 update $X_i \leftarrow X_i + e$;
6 **continue**;
7 **if** *there exist an unallocated item e and an edge $(i,j) \in E$ such that (i,j) is on a directed cycle C and $c_i(e \mid X_j) = 0$* **then**
8 rotate the bundles on the cycle: for each edge $(u,v) \in C$, allocate X_v to agent u;
   `// In particular, i receives` X_j.
9 add e to agent i's bundle;
10 **continue**;
   `// Handling the case where the above two kinds of updates fail.`
11 find a tail strongly connected component S in G;
   `// A tail strongly connected component` S `has no outgoing edge from` S`; in particular, a sink is a tail strongly connected component.`
12 **if** *there are at least $|S|$ unallocated items* **then**
13 allocate each agent S an arbitrary unallocated item;
14 **else**
15 **break**;

Output: $\mathbf{X} = (X_1, \ldots, X_n)$

References

Akrami, H.; Alon, N.; Chaudhury, B. R.; Garg, J.; Mehlhorn, K.; and Mehta, R. 2023. EFX: A Simpler Approach and an (Almost) Optimal Guarantee via Rainbow Cycle Number. In *EC*, 61. ACM.

Amanatidis, G.; Aziz, H.; Birmpas, G.; Filos-Ratsikas, A.; Li, B.; Moulin, H.; Voudouris, A. A.; and Wu, X. 2023. Fair division of indivisible goods: Recent progress and open questions. *Artif. Intell.*, 322: 103965.

Amanatidis, G.; Birmpas, G.; Filos-Ratsikas, A.; Hollender, A.; and Voudouris, A. A. 2021. Maximum Nash welfare and other stories about EFX. *Theor. Comput. Sci.*, 863: 69–85.

Amanatidis, G.; Markakis, E.; and Ntokos, A. 2020. Multiple birds with one stone: Beating 1/2 for EFX and GMMS via envy cycle elimination. *Theor. Comput. Sci.*, 841: 94–109.

Aziz, H.; Li, B.; Moulin, H.; and Wu, X. 2022. Algorithmic fair allocation of indivisible items: a survey and new questions. *SIGecom Exch.*, 20(1): 24–40.

Aziz, H.; Lindsay, J.; Ritossa, A.; and Suzuki, M. 2023. Fair Allocation of Two Types of Chores. In *AAMAS*, 143–151. ACM.

Babaioff, M.; Ezra, T.; and Feige, U. 2021. Fair and truthful mechanisms for dichotomous valuations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 5119–5126.

Barman, S.; Narayan, V. V.; and Verma, P. 2023. Fair Chore Division under Binary Supermodular Costs. In *AAMAS*, 2863–2865. ACM.

Barman, S.; and Verma, P. 2021. Approximating Nash Social Welfare Under Binary XOS and Binary Subadditive Valuations. In *WINE*, volume 13112 of *Lecture Notes in Computer Science*, 373–390. Springer.

Benabbou, N.; Chakraborty, M.; Igarashi, A.; and Zick, Y. 2020. Finding Fair and Efficient Allocations When Valuations Don't Add Up. In *SAGT*, volume 12283 of *Lecture Notes in Computer Science*, 32–46. Springer.

Bérczi, K.; Bérczi-Kovács, E. R.; Boros, E.; Gedefa, F. T.; Kamiyama, N.; Kavitha, T.; Kobayashi, Y.; and Makino, K. 2020. Envy-free Relaxations for Goods, Chores, and Mixed Items. *CoRR*, abs/2006.04428.

Berger, B.; Cohen, A.; Feldman, M.; and Fiat, A. 2022. Almost Full EFX Exists for Four Agents. In *AAAI*, 4826–4833. AAAI Press.

Bogomolnaia, A.; and Moulin, H. 2004. Random matching under dichotomous preferences. *Econometrica*, 72(1): 257–279.

Bu, X.; Song, J.; and Yu, Z. 2023. EFX Allocations Exist for Binary Valuations. *CoRR*, abs/2308.05503.

Caragiannis, I.; Gravin, N.; and Huang, X. 2019. Envy-Freeness Up to Any Item with High Nash Welfare: The Virtue of Donating Items. In *EC*, 527–545. ACM.

Caragiannis, I.; Kurokawa, D.; Moulin, H.; Procaccia, A. D.; Shah, N.; and Wang, J. 2019. The Unreasonable Fairness of Maximum Nash Welfare. *ACM Trans. Economics and Comput.*, 7(3): 12:1–12:32.

Chan, H.; Chen, J.; Li, B.; and Wu, X. 2019. Maximin-Aware Allocations of Indivisible Goods. In *AAMAS*, 1871–1873. International Foundation for Autonomous Agents and Multiagent Systems.

Chaudhury, B. R.; Garg, J.; and Mehlhorn, K. 2020. EFX Exists for Three Agents. In *EC*, 1–19. ACM.

Chaudhury, B. R.; Garg, J.; Mehlhorn, K.; Mehta, R.; and Misra, P. 2021a. Improving EFX Guarantees through Rainbow Cycle Number. In *EC*, 310–311. ACM.

Chaudhury, B. R.; Kavitha, T.; Mehlhorn, K.; and Sgouritsa, A. 2021b. A Little Charity Guarantees Almost Envy-Freeness. *SIAM J. Comput.*, 50(4): 1336–1358.

Ebadian, S.; Peters, D.; and Shah, N. 2022. How to Fairly Allocate Easy and Difficult Chores. In *AAMAS*, 372–380. International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS).

Foley, D. 1967. Resource Allocation and the Public Sector. *Yale Economic Essays*, 45–98.

Garg, J.; and Murhekar, A. 2021. Computing Fair and Efficient Allocations with Few Utility Values. In *SAGT*, volume 12885 of *Lecture Notes in Computer Science*, 345–359. Springer.

Garg, J.; Murhekar, A.; and Qin, J. 2022a. Fair and Efficient Allocations of Chores under Bivalued Preferences. In *AAAI*, 5043–5050. AAAI Press.

Garg, J.; Murhekar, A.; and Qin, J. 2022b. Improving Fairness and Efficiency Guarantees for Allocating Indivisible Chores. *CoRR*, abs/2212.02440.

Halpern, D.; Procaccia, A. D.; Psomas, A.; and Shah, N. 2020. Fair Division with Binary Valuations: One Rule to Rule Them All. In *WINE*, volume 12495 of *Lecture Notes in Computer Science*, 370–383. Springer.

Jahan, S. C.; Seddighin, M.; Javadi, S. M. S.; and Sharifi, M. 2022. Rainbow Cycle Number and EFX Allocations: (Almost) Closing the Gap. *CoRR*, abs/2212.09482.

Kurokawa, D.; Procaccia, A. D.; and Shah, N. 2018. Leximin Allocations in the Real World. *ACM Trans. Economics and Comput.*, 6(3-4): 11:1–11:24.

Li, B.; Li, Y.; and Wu, X. 2022. Almost (Weighted) Proportional Allocations for Indivisible Chores. In *WWW*, 122–131. ACM.

Plaut, B.; and Roughgarden, T. 2020. Almost envy-freeness with general valuations. *SIAM Journal on Discrete Mathematics*, 34(2): 1039–1068.

Viswanathan, V.; and Zick, Y. 2023. Yankee Swap: A Fast and Simple Fair Allocation Mechanism for Matroid Rank Valuations. In *AAMAS*, 179–187. ACM.

Zhou, S.; and Wu, X. 2022. Approximately EFX Allocations for Indivisible Chores. In *IJCAI*, 783–789. ijcai.org.

A Relationship Between Set Functions with Binary Marginals

In this section, we explore the relationship between additive, cancelable, and submodular set functions, and we focus only on set functions with binary marginals. Many results will be used in later sections, and they are interesting observations that are independent of our applications to fair division.

Firstly, it is obvious from definitions that all additive functions are cancelable. Moreover, there exist cancelable set functions with binary marginals that are not additive. An example is given below.

$$\phi(X) = \begin{cases} 5 & \text{when } |X| \geq 5 \\ |X| & \text{otherwise} \end{cases}. \tag{2}$$

Therefore, we have the following theorem.

Theorem 2.1. *The set of binary additive set functions is a proper subset of the set of cancelable set functions with binary marginals.*

Next, we show that, by applying binary marginal property to cancelable functions, any cancelable function with binary marginals is also submodular.

Theorem 2.2. *The set of cancelable set functions with binary marginals is a proper subset of the set of submodular set functions with binary marginals.*

Proof. To show the set containment, we will show that any set function $\phi : M \to \mathbb{Z}_{\geq 0}$ (with binary marginals) that is not submodular cannot be cancelable. Since ϕ is nonsubmodular, there exists $S \subseteq T \subseteq M$ and $e \in M \setminus T$ such that $\phi(S+e)-\phi(S) < \phi(T+e)-\phi(T)$. We initialize $T' = S$ and iteratively add an element from $T \setminus S$ to T'. At the start, we have $\phi(S+e)-\phi(S) = \phi(T'+e)-\phi(T')$. We consider the first time we observe $\phi(S+e)-\phi(S) < \phi(T'+e)-\phi(T')$ after an element f is added to T'. Let U be the state of T' before f is added. Since ϕ has binary marginals, we must have $\phi(S+e)-\phi(S) = \phi(U+e)-\phi(U)$ and $\phi(S+e)-\phi(S) < \phi(U+e+f)-\phi(U+f)$. Combining the two equations and by noting that ϕ is monotone, we must have $0 \leq \phi(U+e)-\phi(U) < \phi(U+e+f)-\phi(U+f)$. Since $U+e+f$ and $U+f$ differ by only one element and ϕ has binary marginals, we must have $0 \leq \phi(U+e)-\phi(U) < \phi(U+e+f)-\phi(U+f) \leq 1$. It must be that $\phi(U+e)-\phi(U) = 0$ and $\phi(U+e+f)-\phi(U+f) = 1$. Thus, $\phi(U+e+f) > \phi(U+f)$ fails to imply $\phi(U+e) > \phi(U)$, so ϕ is not cancelable.

To show the containment is proper, we present an example of submodular set functions (with binary marginals) that is not cancelable. Let $M = \{a, b, c, d\}$ and

$$\phi(X) = \begin{cases} |X|-1 & \text{if } \{a,b,c\} \subseteq X \\ |X| & \text{otherwise} \end{cases}.$$

It is straightforward to check (e.g., by enumerating all cases) that ϕ is submodular. It is not cancelable as $\phi(\{c,d\}) = \phi(\{b,c\}) = 2$ and $\phi(\{a,c,d\}) = 3 > 2 = \phi(\{a,b,c\})$. □

Note that for set functions with non-binary marginals, the containment in Theorem 2.2 does not hold. A simple cancelable set function that is non-submodular is $\phi(X) = 2^{|X|}$.

We list some of the useful properties of cancelable functions, whose proofs are straightforward.

Proposition 2.3. *Let $\phi : M \to \mathbb{Z}_{\geq 0}$ be a cancelable function.*

1. *For any $S, T \subseteq M$ and any $e \in M \setminus (S \cup T)$, if $\phi(S) = \phi(T)$, then $\phi(S+e) = \phi(T+e)$.*
2. *For any $S, T \subseteq M$ and any $U \subseteq M$ with $U \cap (S \cup T) = \emptyset$, if $\phi(S) = \phi(T)$, then $\phi(S \cup U) = \phi(T \cup U)$.*

Finally, we prove some properties for submodular functions. By Theorem 2.2, cancelable functions with binary marginals also satisfy these properties.

Proposition 2.4. *Let $\phi : M \to \mathbb{Z}_{\geq 0}$ be a submodular function with binary marginals.*

1. *For any $S, T \subseteq M$, $\phi(S)+\phi(T) \geq \phi(S \cup T)+\phi(S \cap T)$. In particular, $\phi(S)+\phi(T) \geq \phi(S \cup T)$.*
2. *If $\phi(S) = 1$ holds for some S with $|S| \geq 2$, there exists $e \in S$ such that $\phi(S-e) = 1$.*

Proof. 1 is a well-known alternative definition for submodularity. For 2, if $\phi(S-e) = 0$ for any $e \in S$, this means every subset of S with size $|S|-1$ has value 0. By monotonicity of ϕ, every element of S has value 0. Then, $\phi(S) = 1$ violates the submodularity. □

B Missing Proofs in Section 3

Proof of Lemma 3.3: We show that the allocation $\mathbf{X}$ is EFX by mathematic induction. At the beginning of Phase 2, we must have that the partial allocation is EFX since each agent only receives items that cost 0 to her. In the following, we assume that the (partial) allocation is EFX at the beginning of some round t. We show that the (partial) allocation is still EFX for all agents at the end of round t.

According to the algorithm, we only have to consider the case under the if condition (in line 9) that we reallocate some items. Let X_i, X_j be the bundles that agents i, j hold at the beginning of round t, respectively. During the round, we reallocate a subset of items $S \subseteq X_j$ to agent i and assign an item $e \in M^+$ to agent j. We must have $c_i(e) = 1$ otherwise the if-condition does not hold. We show that at the end of the round, the new allocation $\mathbf{X}'$ is EFX for all agents, while $X'_i = X_i \cup S$, $X'_j = X_j \setminus S + e$ and $X'_k = X_k$ for all $k \in N \setminus \{i, j\}$. Before we give the proof, we first show the following claim.

Claim B.1. *If the if-condition from Line 9 to Line 12 is executed, we must have $c_i(X_i) = c_j(X_j)$. After the execution, we have $X_j \setminus S \subseteq M^+$.*

Proof. For the first statement, we assume otherwise that $c_j(X_j) \geq c_i(X_i) + 1$. Then we have $c_i(X_i + e) = c_i(X_i) + 1 \leq c_j(X_j) \leq c_i(X_j)$, where the last inequality holds since the (partial) allocation minimizes social cost. In other words, agent i is envy-free towards j after allocating e to i, which is a contradiction. Hence the only case that agent i is not EFX towards j after assigning item e is that $c_i(X_i) = c_j(X_j) = c_i(X_j)$. Following the minimum social cost property, for any item $e' \in X_j$, we have $c_i(e') = c_j(e')$. Hence we have $S \subseteq M^0$ and $X_j \setminus S \subseteq M^+$. □

Given Claim B.1, we are ready to show the allocation is EFX for all agents at the end of round t. We show the property holds for agents i, j and any $k \neq i, j$ individually.

- For any $k \neq i, j$, agent i is EFX towards j and k. Note that during the reallocation, we only reallocate those items that cost 0 to agent i, i.e., $c_i(S) = 0$. Hence we have $c_i(X_i') = c_i(X_i \cup S) = c_i(X_i)$. Note that the allocation $\mathbf{X}$ is EFX for agent i and $X_k = X_k'$ for any $k \neq i, j$. We have agent i is EFX towards any agent k at the end of the round. As for the envy between i and j, we have $c_i(X_j') = c_i(X_j \setminus S + e) = c_i(X_j) + 1$, agent i is EFX towards agent j at the end of the round.
- For any $k \neq i, j$, agent j is EFX towards i and k. Following Claim B.1, agent j has the minimum bundle cost among all agents at the beginning of the round. In other words, agent j is envy-free towards any agent $k \neq j$ since $c_j(X_j) \leq c_k(X_k) \leq c_j(X_k)$, where the second inequality holds since the (partial) allocation minimizes social cost. Combining $X_j \setminus M^+$ and $e \in M^+$, we have $c_j(X_j' - e') = c_j(X_j) \leq c_j(X_k)$ for any $e' \in X_j'$ and any $k \neq j$.
- For any $k \neq i, j$, agent k is EFX towards i and j. Due to the monotonicity of c_k, we have $c_k(X_i') \geq c_k(X_i)$, agent k is EFX towards i at the end of the round. Next, we show that k is EFX towards j. We first claim that $c_k(X_k) \leq c_i(X_i) + 1$. Assume otherwise $c_k(X_k) \geq c_i(X_i) + 2$, we consider the last time that we assign an item $e' \in M^+$ to agent k. We must have $c_k(X_k - e') \geq c_i(X_i) + 1$, which contradicts the fact that k holds a bundle with minimum cost. Hence we consider the cases that $c_k(X_k) = c_i(X_i)$ and $c_k(X_k) = c_i(X_i) + 1$. For both cases k is EFX towards j since $c_k(X_k) \leq c_i(X_i)+1 = c_j(X_j') \leq c_k(X_j')$, where the equality follows from Claim B.1 and the fact that $e \in M^+$.

In conclusion, we show that the (partial) allocation is EFX for all agents, at the end of the round. Note that in each round we allocate one item in M^+ and the algorithm terminates Phase 2 after $|M^+|$ rounds. Hence upon the running of Algorithm 1, it computes an EFX allocation. ■

Lemma B.1. *Algorithm 1 runs in $O(nm^2)$ time.*

Proof. Algorithm 1 starts from a partial allocation $\mathbf{X}^0$ with zero social cost, which can be computed in $O(mn)$ times (there are at most m items in M^0 and each item can be allocated in $O(n)$ time). The algorithm runs in rounds in phase 2 and there are at most m rounds to allocate items in M^+. In each round, determining the agent i who receives item e can be done in $O(\log n)$ time, and determining whether i is EFX towards other agents can be done in $O(mn)$ time. The reallocation in each round can be done in $O(m)$ time since there are at most m items in X_j. In conclusion, Algorithm 1 runs in $O(nm^2)$ time. □

Non-existence of EFX and PO

In this section, we complete our result by exploring the non-existence of EFX and PO allocations in the additive setting. We show that even extending our result to ternary instances (where the cost of each item can only be $0, 1, 2$) is impossible. Whether bivalued instances (another generalization of binary instances) admit EFX and PO allocations for the general number of agents remains open.

Definition B.2 (Ternary Instances)**.** *An instance is called* ternary *if for each agent $i \in N$ and any item $e \in M$ we have $c_i(e) \in \{0, 1, 2\}$.*

Example B.3. *Consider an instance with $n = 2$ agents and $m = 3$ items. Both agents have ternary cost functions, that is, $c_i(e) \in \{0, 1, 2\}$ for all $i \in N$ and $e \in M$. The costs are shown in Table 1. Note that any PO allocation should assign e_2 to agent 2 and e_3 to agent 1. Hence PO allocations can only be $X_1 = \{e_1, e_3\}, X_2 = \{e_2\}$ or $X_1 = \{e_3\}, X_2 = \{e_1.e_2\}$. None of these PO allocations is EFX since the agent who received item e_1 still envies another agent even after removing the item with zero cost.*

	e_1	e_2	e_3
agent 1	2	1	0
agent 2	2	0	1

Table 1: Instance showing that EFX and PO allocations do not exist for ternary chores.

C Missing Proofs in Section 4

Proof of Lemma 4.2: We prove by induction that $c_i(A_j) = t$ for each pair of i and j after t while-loop iterations. The base step for $t = 0$ is trivial. Suppose the claim holds for t. We will show it holds for $t + 1$. Let $(A_1, \ldots, A_n)$ be the allocation before the $(t + 1)$-th iteration. By induction hypothesis, $c_i(A_j) = t$ for every pair of i and j. For each item e allocated in the $(t+1)$-th iteration, we have $c_i(e \mid A_i) = 1$ for each agent i. By Property 1 in Proposition 2.3, this also implies $c_i(e \mid A_j) = 1$ for every other agent j. Thus, $c_i(A_j) = t + 1$ for every pair of i and j after the $(t + 1)$-th iteration. ■

Proof of Theorem 4.8: Consider n agents with $5n$ items where each agent's cost function is given by Equation (2). It is easy to check that the only EFX allocation gives exactly 5 items to each agent. However, this allocation is Pareto-dominated by the allocation that allocates all items to a single agent. ■

D Submodular Cost Functions with Binary Marginals

Theorem D.1. *For submodular cost functions with binary marginals, there exists an allocation $\mathbf{X}$ that is either EFX or 2-EF. In addition, such an allocation can be computed in polynomial time.*

Notice that a 2-EF allocation is always 2-EFX. We have the following corollary.

Corollary D.2. *For submodular cost functions with binary marginals, there exists a 2-EFX allocation and it can be computed in polynomial time.*

The proof of Theorem D.1 is mostly based on algorithms in the previous sections.

Proof of Theorem D.1. Let $M_1 = \{e \in M \mid c_i(e) = 1 \text{ for all } i\}$. We consider two cases.

Case 1: $|M_1| < n$. In this case, we will run Algorithm 2. Note that Phase 1 will be skipped. As a result, each d_i in the algorithm will be c_i, which is submodular. By Lemma 4.4 and Remark 4.7, an EFX allocation will be output.

Case 2: $|M_1| \geq n$. In this case, we will initialize the allocation $(X_1, \ldots, X_n)$ such that each X_i contains exactly one item in M_1. The current partial allocation is clearly EF. Next, we will implement the while-loop in Algorithm 3. Lastly, for the unallocated items, we allocate them arbitrarily such that each agent receives at most one extra item.

Since we have $c_i(X_j) = 1$ for any i and j before the while-loop and the algorithm never removes items from a bundle, it holds that $c_i(X_j) \geq 1$ for any i and j for the final allocation. The allocation remains EF during the while-loop, and the envy-freeness can only be broken at the last step. By the binary marginal property, if i envies j, it must be that $c_i(X_i) = w + 1$ and $c_i(X_j) = w$ for some w. We have $c_i(X_i) \leq 2 \cdot c_i(X_j)$ since we have seen that $w \geq 1$. □